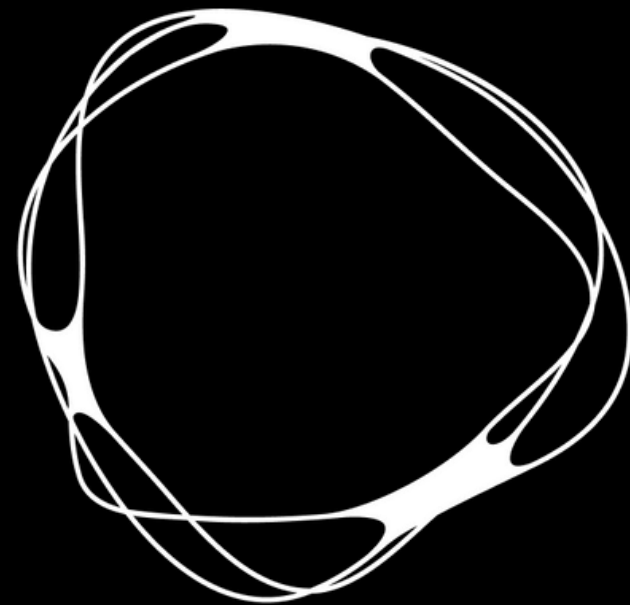




STADS

**Students' Association for
Data Analytics & Statistics**

SQL – Kurs 2026

Kursleiter: Milan Koncz

in Kooperation mit

Prof. Dr. Martin Schlather

Lehrstuhl für Stochastik und ihre Anwendungen

Prof. Dr. Leif Döring

Lehrstuhl für Stochastik

Agenda

- **Einführung**
- **Auswahl von Einträgen**
- **Zählen von Einträgen, bedingte Abfragen**
- **Funktionen**
- **Verbinden von Tabellen**
- **Ausblick**



Einführung



SQL

- SQL = Structured Query Language
- MySQL != PostgreSQL != SQLite
- Sprache für Datenbanken

**Running a national
government database
on SQL lite**



**Running a national
government database
on MS Access**



**Running a national
government database
on MS Excel**



SQL

- Bitte installiert SQLite <https://sqlitebrowser.org/dl/> auf eurem PC und startet das Programm.
- Ladet die Testdatenbank runter!



<https://limewire.com/d/oQt3X#UTV0VJX5CN>

Was ist eine relationale Datenbank?

- Daten werden in Tabellen mit Zeilen und Spalten gespeichert
- Tabellen sind über Schlüssel miteinander verbunden
- Verwaltung und Abfragen erfolgen meist mit SQL
- Fast wie Excel, nur cooler

Was ist eine relationale

The screenshot shows an Excel spreadsheet with three tables of acoustic data. The entire spreadsheet is crossed out with a large red 'X'.

Table 1: c:\file directory\acoustic data\Site 10\20160517

ID	EPFU	LANO	LABO	LACI	MYGR	NYHU	PESU	UNKN	LOW	MID	MYOTIS
N	70	12	210	7	3	83	19	11	91	319	
%	16.87	2.89	50.60	1.69	0.72	25.00	4.58	2.65	21.93	76.87	1.5
MLE (p)	0.000001	0.000001	0.000001	0.000001	0.000001	0.000001	0.000001				

Table 2: c:\file directory\acoustic data\Site 10\20160518

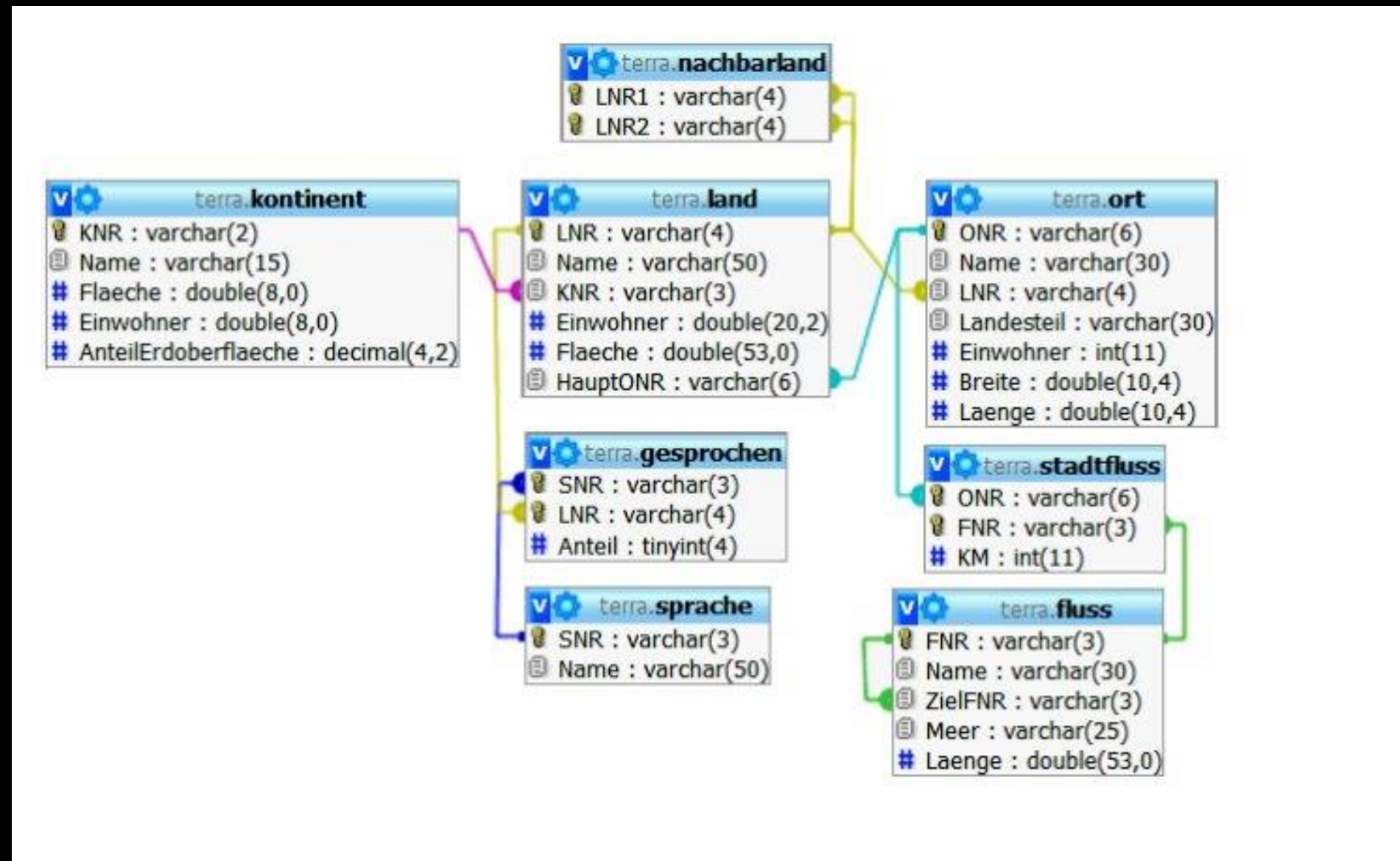
ID	LABO	MYGR	MYHU	NYHU	PESU	MID	MYOTIS	Total
N	10	2	1	6	2	18	3	21
%	47.62	9.52	4.76	28.57	9.52	85.71	14.29	
MLE (p)	0.000001	0.000001	0.088437	0.000121	0.018334			

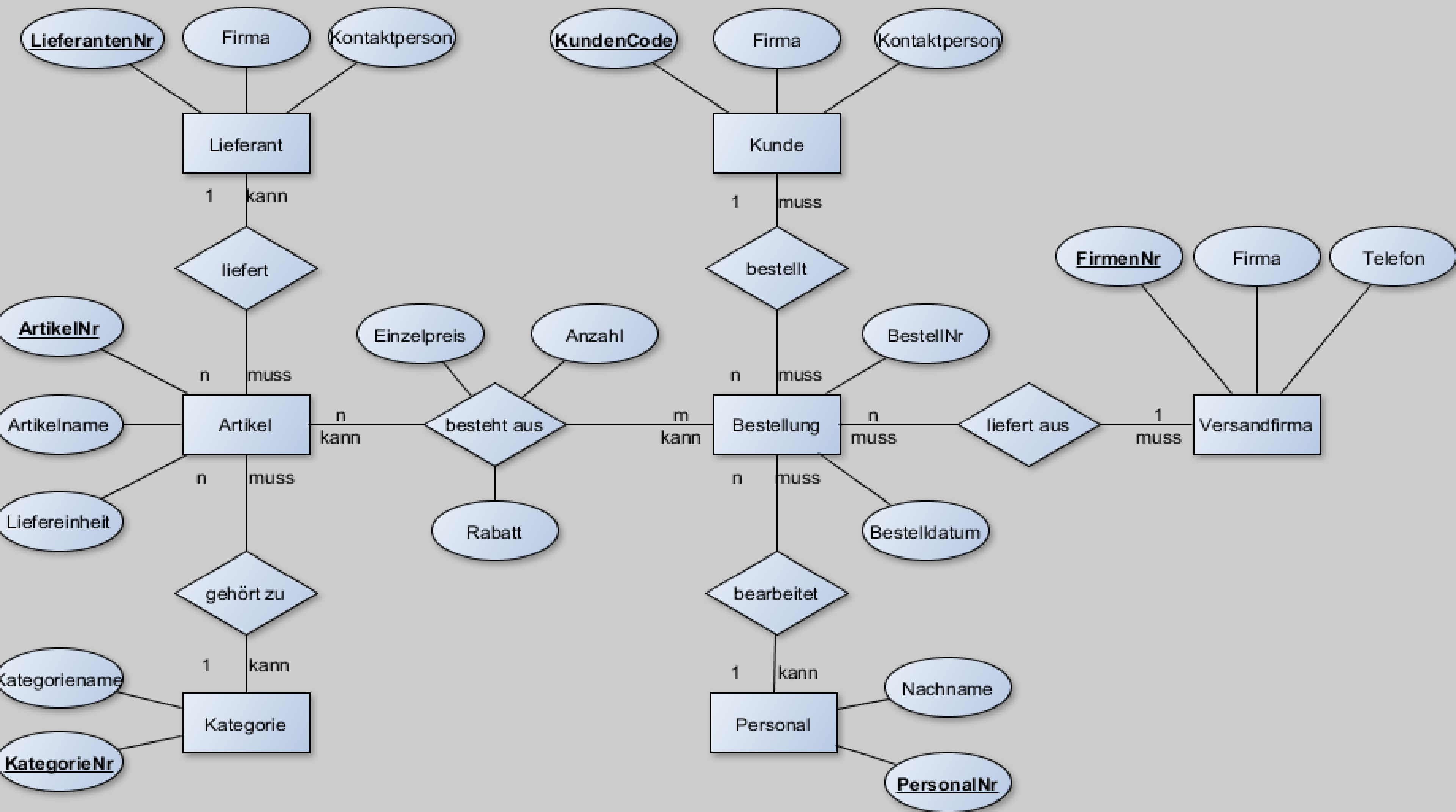
Table 3: c:\file directory\acoustic data\Site 11\20160517

ID	LABO	MYGR	MYHU	NYHU	PESU	MID	MYOTIS	Total
N	10	2	1	6	2	18	3	21
%	47.62	9.52	4.76	28.57	9.52	85.71	14.29	
MLE (p)	0.000001	0.000001	0.088437	0.000121	0.018334			

Was ist eine relationale Datenbank?

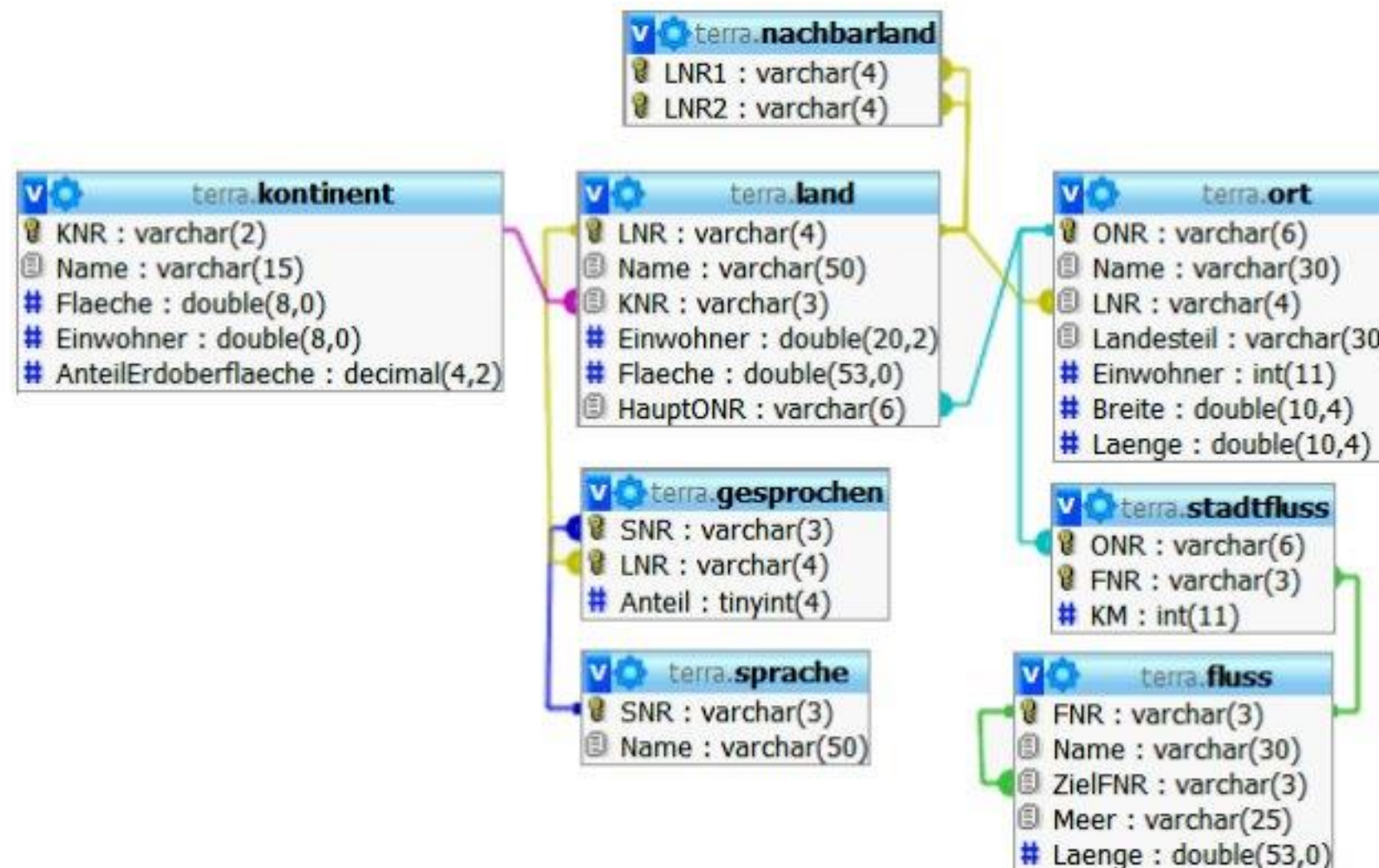
- "Eine Gruppe zusammenhängender Tabellen"





Was ist eine relationale Datenbank?

- "Eine Gruppe zusammenhängender Tabellen"



Wofür wird SQL benötigt?

Studenten

Matrikelnummer	Name
1519315	Jonas Busch
1523456	Lukas Christ

Vorlesung

Vorlesung Code	<u>Pers.nummer</u>
FIN510	15
OPM504	28
OPM310	7

Belegung

Matrikelnummer	Vorlesung
1519315	FIN510
1523456	OPM504
1523456	OPM310

Professoren

<u>Pers.nummer</u>	Name
15	A. Schmidt
7	D. Mueller
28	A. Vogel

Beispielaufgaben im Alltag

- Wie viele Kunden leben in Mannheim?
- Was waren die beliebtesten Songs 2024?
- Zu welcher Inselgruppe gehört Aust-Vagoey?
- Welche Art von Schokolade wird in Mannheim produziert?



Allgemeine Infos

- "--" Macht Kommentar. Alternativ /* ... */
- Befehle werden durch Semicolon ; getrennt
- Für SQLite: Einzelner Befehl wird durch shift + F5 ausgeführt

Allgemeine Infos

- SQL-Befehle sind nicht Groß-/Kleinschreibungs-sensitiv
- Konvention: Schlüsselwörter GROSS, Tabellen/Spalten klein.

Grundstruktur einer Abfrage:

```
SELECT spalte1, spalte2  
FROM tabelle  
WHERE bedingung  
ORDER BY spalte;
```




Auswahl von Einträgen

Die SELECT Anweisung

Alle Spalten anzeigen (*)

```
SELECT * FROM Berg;
```

Bestimmte Spalten auswählen

```
SELECT Name, Hoehe FROM Berg;
```

Anzahl der Zeilen begrenzen (LIMIT)

```
SELECT Name, Hoehe FROM Berg LIMIT 10;
```

Eine Spalte anzeigen lassen

Syntax	Beispiel
<code>SELECT (Spalte) FROM (Tabelle);</code>	<code>SELECT Name FROM Stadt;</code>

	Name
1	Aalborg
2	Aarhus
3	Abadan
4	Abidjan
5	Abu_Dhabi
6	Accra
7	Addis_Abeba

Mehrere Spalten anzeigen lassen

Syntax	Beispiel
SELECT (spalte1), (spalte2) FROM (tabelle);	SELECT Name, Gebirge, Hoehe FROM Berg;
SELECT * FROM (tabelle);	SELECT * FROM Berg;

	Name	Gebirge	Hoehe
1	Aconcagua	Anden	6958
2	Allahu_Akbar	Sagrosgebirge	2804
3	Baghe_Bala	Sagrosgebirge	3775
4	Bahraseman	Sagrosgebirge	3886
5	Bjelucha	Altai	4506
6	Blue_Mountain_Peak	Blue_Mountains	2256
7	Borah_Peak	Rocky_Mountains	3859

	B_ID	Name	Gebirge	Hoehe	Jahr	Laenge	Breite
1	1	Aconcagua	Anden	6958	NULL	-70	-32.5
2	2	Allahu_Akbar	Sagrosgebirge	2804	NULL	NULL	NULL
3	3	Baghe_Bala	Sagrosgebirge	3775	NULL	NULL	NULL
4	4	Bahraseman	Sagrosgebirge	3886	NULL	NULL	NULL
5	5	Bjelucha	Altai	4506	1933	86.4	49.48
6	6	Blue_Mountain_Peak	Blue_Mountains	2256	NULL	-76.4	18.1
7	7	Borah_Peak	Rocky_Mountains	3859	NULL	-114	44.1

Aufgaben

Aufgaben:

1. Gebe die Namen aller Kontinente aus.
2. Gebe die Fläche aller Seen aus.
3. Gebe die Preise aller Flüge aus.

Lösungen

Lösungen:

1. `SELECT Name FROM Kontinent;`
2. `SELECT Flaeche FROM See;`
3. `SELECT Preis FROM Flug;`

	Name
1	Afrika
2	Amerika
3	Asien
4	Australien
5	Europa

	Flaeche
1	<i>NULL</i>
2	5347
3	<i>NULL</i>
4	62155
5	40
6	100
7	31500

	Preis
1	790
2	790
3	770
4	770
5	1790
6	1790

Aufgaben

Aufgaben:

1. Lass dir alle Flugnummern ausgeben.
2. Lass dir alle Flugnummern und den Abflugshafen ausgeben.
3. Lass dir Alles zum Thema Flug ausgeben.

Aufgaben & Lösungen

Aufgaben:

1. Lass dir alle Flugnummern ausgeben.
2. Lass dir alle Flugnummern und den Abflugshafen ausgeben.
3. Lass dir Alles zum Thema Flug ausgeben.

Lösungen:

1. `SELECT Flug_ID FROM Flug;`
2. `SELECT Flug_ID, AbflugFH_ID FROM Flug;`
3. `SELECT * FROM Flug;`

DISTINCT

Zeigt nur verschiedene Einträge einer Spalte an.

Syntax	Beispiel
<code>SELECT DISTINCT (spalte) FROM (tabelle);</code>	<code>SELECT DISTINCT Wuestenart FROM Wueste;</code>

Aufgaben:

1. Gebe dir alle verschiedenen Gebirge aus.
2. Gebe dir alle verschiedenen Inselgruppen aus.

Lösungen - DISTINCT

Aufgaben:

1. Gebe dir alle verschiedenen Gebirge aus.
2. Gebe dir alle verschiedenen Inselgruppen aus.

Lösungen:

1. `SELECT DISTINCT Gebirge FROM Berg;`
2. `SELECT DISTINCT Inselgruppe FROM Insel;`

Rechenoperatoren, Alias

- Benenne lange/ungünstige Spaltennamen um
- Rechenoperatoren im Query: Add (+), Sub (-), Mult (*), Div (/)

Syntax	Beispiel
SELECT (spalte) AS (alias) FROM (tabelle);	SELECT Name AS Titel FROM Berg;
SELECT (spalte) [Rechenoperation] AS (alias) FROM (tabelle);	SELECT Name, Hoehe / 1000.0 AS Hoehe_km FROM Berg;



After you learn
SQL and Tableau



Zählen von Einträgen
Bedingte Abfragen

Count ()

Diese Funktion zählt die Einträge einer Tabelle.

Syntax	Beispiel
Zählt alle Einträge/Zeilen einer Tabelle	
SELECT COUNT(*) AS (Name) FROM (TABELLE);	SELECT COUNT(*) AS Anzahl FROM Insel;
Zählt alle Einträge/befüllte Zeilen einer Spalte	
SELECT COUNT((SPALTE)) FROM (TABELLE);	SELECT COUNT(Inselgruppe) FROM Insel;
Zählt alle verschiedenen Einträge/Zeilen einer Spalte	
SELECT COUNT(DISTINCT (SPALTE)) FROM (TABELLE);	SELECT COUNT(DISTINCT Inselgruppe) FROM Insel;

Aufgaben – COUNT()

Aufgaben:

1. Wie viele Länder sind in dem Datensatz enthalten? Gebe die Antwort als Anzahl aus.
2. Wie viele Flüsse sind in dem Datensatz enthalten?
3. Wie viele Flusslängen sind bekannt?
4. Wie viele unterschiedliche Gebirge sind bekannt?

Lösungen – COUNT()

Aufgaben:

1. Wie viele Länder sind in dem Datensatz enthalten?
2. Wie viele Flüsse sind in dem Datensatz enthalten?
3. Wie viele Flusslängen sind bekannt?
4. Wie viele unterschiedliche Gebirge sind bekannt?

Lösungen:

1. `SELECT COUNT(*) AS Anzahl FROM Land;`
2. `SELECT COUNT(*) FROM Fluss;`
3. `SELECT COUNT(Flusslaenge) FROM Fluss;`
4. `SELECT COUNT(DISTINCT Gebirge) FROM Berg;`

WHERE - Bedingung

Syntax	Beispiel
WHERE für erste Bedingung, alle weiteren mit AND	
SELECT (SPALTE) FROM (TABELLE) WHERE (BEDINGUNG) AND (BEDINGUNG);	SELECT Name FROM Stadt WHERE Einwohner > 1100000;
Mögliche Operatoren:	
= equal <> not equal < less than	> greater than <= less than or equal to >= greater than or equal to

WHERE - Wissenswertes

- Bedingung bei VarChar-Attributen (Reiner Text)

Syntax	Beispiel
SELECT (SPALTE) FROM (TABELLE) WHERE (SPALTE) = "(BEDINGUNG)";	SELECT Einwohner FROM Stadt WHERE Name = "Aalborg";

- Hier auf Groß- und Kleinschreibung achten!
- Hochkommata sind auch möglich: Name = 'Aalborg'

Aufgaben - WHERE

Aufgaben:

1. Lass dir alle Informationen zu Flügen ausgeben, die (echt) teurer als 1.500 Euro gewesen sind, aber billiger (gleich) als 1.700 Euro.
2. Lass dir die Namen aller Städte anzeigen, die (echt) mehr als 100.000 Einwohner haben und dessen Längengrad (echt) größer 10 ist.
3. Wie groß ist die Fläche Afrikas?
4. In welchem Gebirge liegt die Zugspitze?

Lösungen - WHERE

Lösungen:

1. `SELECT * FROM Flug WHERE Preis > 1500 AND Preis <= 1700;`
2. `SELECT Name FROM Stadt WHERE Einwohner > 100000 AND Laenge > 10;`
3. `SELECT FLAECHE FROM Kontinent WHERE Name = "Afrika";`
4. `SELECT Gebirge FROM Berg WHERE Name = 'Zugspitze';`

WHERE ... OR ...

Falls nur eine Bedingung erfüllt sein muss.

Syntax	Beispiel
SELECT (SPALTE) FROM (TABELLE) WHERE (BEDINGUNG1) OR (BEDINGUNG2);	SELECT * FROM Stadt WHERE Name = 'Graz' OR Name = 'Aalborg';
Bei mehreren Bedingungen(>2) müssen Klammern () entsprechend der Logik gesetzt werden.	
SELECT * FROM Stadt WHERE (Name = 'Graz' OR Name = 'Aalborg') AND Einwohner >= 200000;	

Aufgaben – WHERE ... OR

Aufgaben:

1. Lass dir alle Inseln ausgeben, die zu den "Kanal-Inseln" oder den "Lofoten" gehören.
2. Lass dir alle Flugnummern (FlugNr) ausgeben, deren Flugpreis 790 Euro oder 1.180 Euro sind.

Lösungen – WHERE ... OR

Aufgaben:

1. Lass dir alle Inseln ausgeben, die zu den "Kanal-Inseln" oder den "Lofoten" gehören.
2. Lass dir alle Flugnummern (FlugNr) ausgeben, deren Flugpreis 790 Euro oder 1.180 Euro sind.

Lösungen:

1. `SELECT Name FROM Insel WHERE Inselgruppe = 'Kanal-Inseln' OR Inselgruppe = 'Lofoten'; -- 4 Zeilen`
2. `SELECT FlugNr FROM Flug WHERE Preis = 790 OR Preis = 1180; --4 Zeilen`

BETWEEN

Ermöglicht in einer Bedingung auf ein Intervall einzuschränken.

Altes Beispiel	Ist äquivalent zu
<pre>SELECT * FROM Flug WHERE Preis >= 1500 AND Preis <= 1700;</pre>	<pre>SELECT * FROM Flug WHERE Preis BETWEEN 1500 AND 1700;</pre>

Hinweis: Ränder sind inklusive!

IN

Funktioniert wie OR für eine Spalte.

Altes Beispiel	Ist äquivalent zu
<pre>SELECT * FROM Stadt WHERE Name = 'Graz' OR Name = 'Aalborg';</pre>	<pre>SELECT * FROM Stadt WHERE Name IN ('Graz', 'Aalborg');</pre>

Aufgaben – BETWEEN/IN

Aufgaben:

1. Lass dir alle Berge in den Anden und dem Altai Gebirge ausgeben mithilfe des IN Operators.
2. Lass dir alle Wüstennamen ausgeben mit Fläche zwischen 10.000 und 40.000.
3. Lass dir alle Wüstennamen ausgeben mit Fläche zwischen 10.000 und 40.000, die entweder eine Sandwüste oder eine Steinwüste sind.

Lösungen – BETWEEN/IN

1. `SELECT Name FROM Berg WHERE Gebirge IN ('Anden','Altai');` --6 Zeilen
2. `SELECT Name FROM Wueste WHERE Flaeche BETWEEN 10000 AND 40000;` --15 Zeilen
3. `SELECT Name FROM Wueste WHERE Flaeche BETWEEN 10000 AND 40000 AND Wuestenart IN ('Stein', 'Sandwueste');` --8 Zeilen

IS (NOT) NULL

Filtert auf (nicht) fehlende Werte.

Fehlende Werte	Nicht fehlende Werte
<pre>SELECT COUNT(*) FROM Wueste WHERE Flaeche IS NULL;</pre>	<pre>SELECT COUNT(*) FROM Wueste WHERE Flaeche IS NOT NULL;</pre>

	Count(*)
1	5

	Count(*)
1	25

Häufiger Fehler: *WHERE Flaeche = NULL*

(NOT) LIKE

Filtert Attribute nach einer bestimmten Eigenschaft.

Syntax	Beispiel
% - ersetzt mehrere Stellen	"B%"
_ - ersetzt exakt eine Stelle	"_t%"
SELECT (Spalte) FROM (Tabelle) WHERE LOWER(Spalte) LIKE (Bedingung);	SELECT Name FROM Wueste WHERE LOWER(Name) LIKE '%i';

Hinweis: LOWER(...) macht Suche unabhängig von Groß-/Kleinschreibung

'%i' bedeutet: endet auf i

'A%' bedeutet: beginnt mit A

ORDER BY

- Sortieranweisung am Ende der Query

Syntax	Beispiel
SELECT (SPALTE1) [...] ORDER BY (SPALTE1) (DESC/ASC)	SELECT Name, Hoehe FROM Berg ORDER BY Land ASC, Hoehe DESC;

- ASC = Ascending
- DESC = Descending
- Sekundärsortierung auch möglich



(Aggregations-) Funktionen



SUM/AVG/MAX/MIN

Es gibt Funktionen, die die Summe, den Durchschnitt oder den maximalen/minimalen Wert einer Spalte ausgeben.

Beispiele	
SELECT SUM(Einwohner) FROM Stadt;	Die Summe an Einwohnern aller Städte
SELECT AVG(Einwohner) FROM Stadt;	Die durchschnittliche Anzahl an Bewohnern für alle Städte
SELECT MAX(Einwohner) FROM Stadt;	Die maximale Einwohnerzahl

Aufgaben - Funktionen

Aufgaben:

1. Gib den durchschnittlichen Preis aller Flüge aus, sowie die größte AbflugFH_ID.
2. Gib den größten Berg aus, der vor 1800 entdeckt wurde und dessen Längengrad größer als Null ist.
3. Gib die durchschnittliche Tiefe der Seen aus, die (echt) tiefer als 30 sind und maximal eine Fläche von 2000 besitzen.

Lösungen - Funktionen

1. `SELECT AVG(Preis) FROM Flug; --ca 1.113,95€`
`SELECT MAX(AbflugFH_ID) FROM Flug; --10`
2. `SELECT Name, MAX(Hoehe) FROM Berg WHERE Jahr < 1800 AND Laenge > 0; --Montblanc`
3. `SELECT AVG(Tiefe) FROM See WHERE Tiefe > 30 AND Flaeche <= 2000; --191,07m`

GROUP BY

Wird zum gruppieren von Daten verwendet, meist im Zusammenhang mit Funktionen.

Intuition: Aus vielen Zeilen einer Gruppe wird eine einzige Zeile

Syntax	Beispiel
SELECT (Spalte) FROM (Tabelle) GROUP BY (Spalte);	SELECT Inselgruppe, SUM(Flaeche) AS Summe_Flaeche FROM Insel WHERE Inselgruppe IS NOT NULL AND Flaeche IS NOT NULL GROUP BY Inselgruppe;

HAVING

Wird für bedingte Abfragen von gruppierten Daten verwendet.

Syntax	Beispiel
SELECT (Spalte) FROM (Tabelle) GROUP BY (Spalte) HAVING (Ausdruck);	SELECT Inselgruppe, SUM(Flaeche) AS Summe_Flaeche FROM Insel WHERE Inselgruppe IS NOT NULL AND Flaeche IS NOT NULL GROUP BY Inselgruppe HAVING Summe_Flaeche > 100;

HAVING vs WHERE

- WHERE filtert Zeilen vor der Gruppierung
- HAVING filtert Gruppen nach der Gruppierung
- HAVING wird oft mit COUNT, SUM, AVG, MAX oder MIN genutzt

WHERE vs. HAVING

- Filters by each row
- Processed before any grouping
- Cannot have aggregate functions
- Can be used in SELECT, INSERT, UPDATE, DELETE statements
- Written before GROUP BY clause

```
SELECT *  
FROM table  
WHERE column1 >= condition;
```

- Filters by each group
- Processed after any grouping
- Can have aggregate functions
- Can only be used in SELECT statements
- Written after GROUP BY clause

```
SELECT *  
FROM table  
GROUP BY column2  
HAVING MIN(column1) >= condition;
```

clauses that filter data based on conditions

Reihenfolge

1. SELECT
2. FROM / JOIN
3. WHERE
4. GROUP BY
5. HAVING
6. ORDER BY
7. LIMIT

Aufgaben

Aufgaben – GROUP BY/ORDER BY/HAVING:

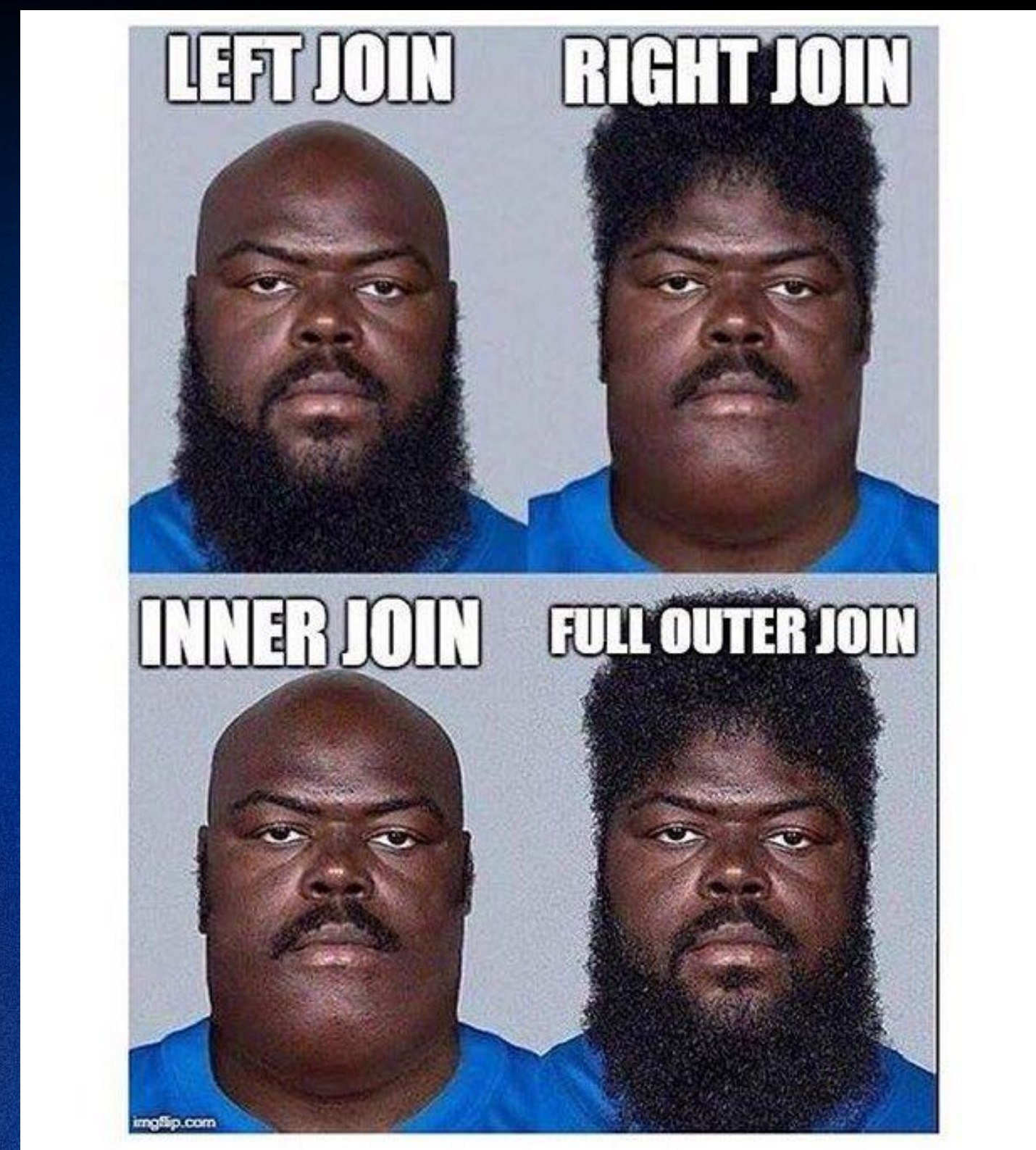
1. Gib für jeden Abfahrt Flughafen die durchschnittlichen Ticketpreise an und sortieren sie diese von teuer nach billig.
2. Ermittel für jedes Gebirge die Höhe des höchsten Berges an.
3. Gib alle Regierungsformen an und zusätzlich die summierte Fläche, die von der jeweiligen Regierungsform geführt wird.
4. Welche Landesteile (Namen) erscheinen in der Tabelle “Landesteil” mehrmals?

Lösungen

1. `SELECT AbflugFH_ID, AVG(Preis) AS Durch_Preis FROM Flug GROUP BY AbflugFH_ID ORDER BY Durch_Preis DESC; --ID:10, AVG:1666.67`
2. `SELECT Gebirge, MAX(Hoehe) as Max_Hoehe FROM Berg GROUP BY Gebirge; -- Anden, 6958m`
3. `SELECT Staatsform, SUM(Flaeche) AS Summe_Flaeche FROM Land GROUP BY Staatsform; --Republik, 3887477.17m2`
4. `SELECT Name, COUNT(*) AS Anzahl FROM Landesteil GROUP BY Name HAVING Anzahl >= 2; --3 Zeilen`

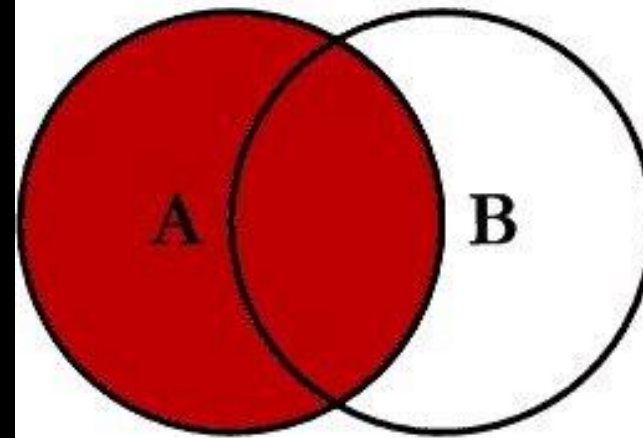


Verbinden von Tabellen

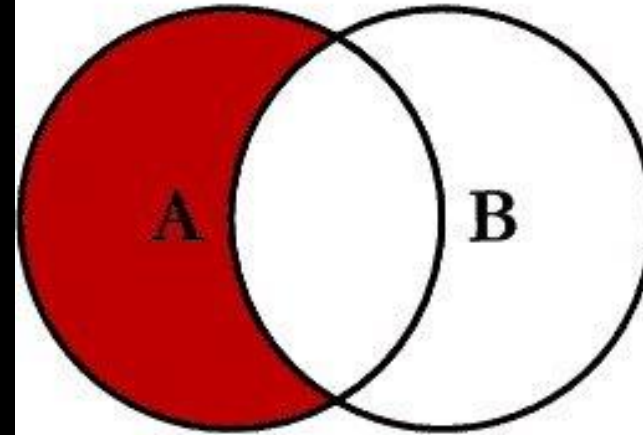


Joins

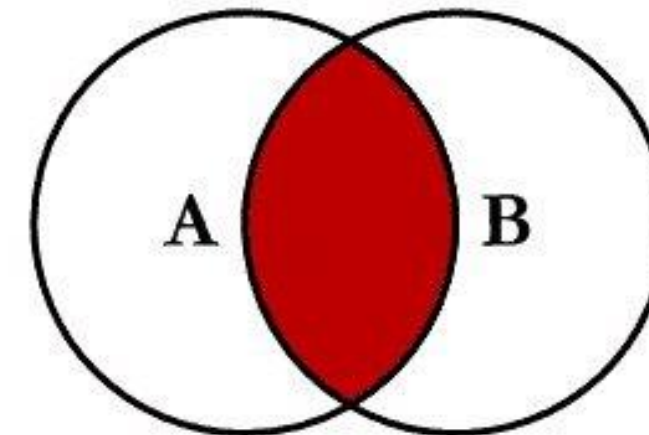
SQL JOINS



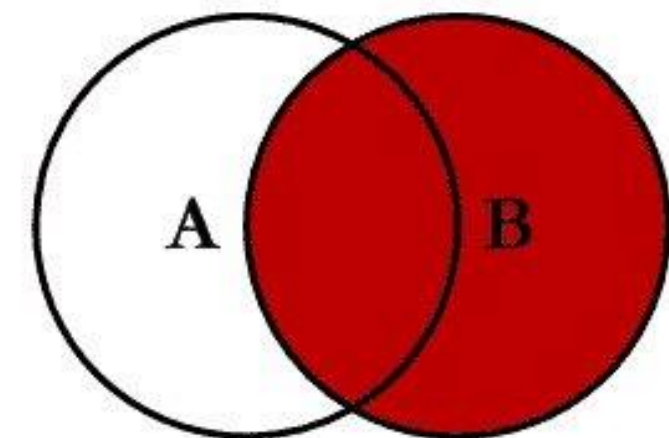
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



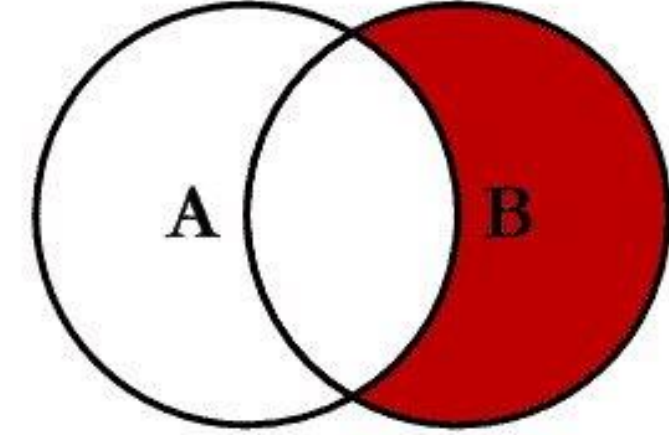
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```



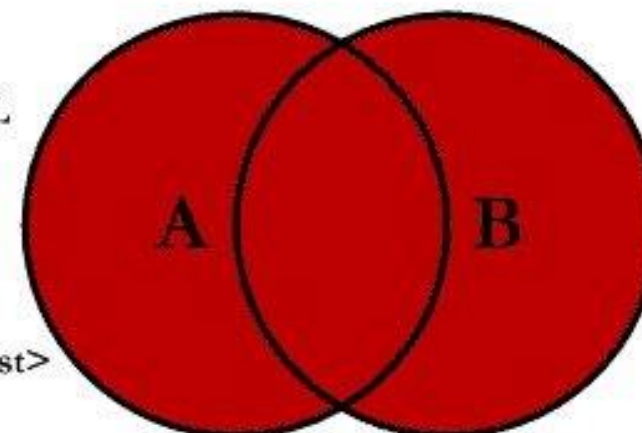
```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



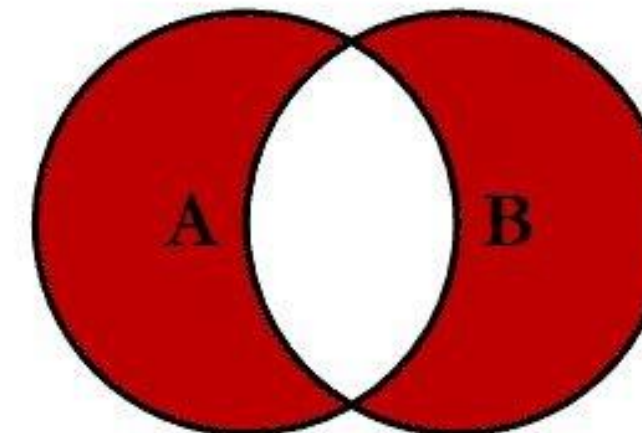
```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```

INNER JOIN

Kombiniert mehrere Tabellen bezüglich eindeutiger Spalten.

Syntax	Beispiel
<pre>SELECT * FROM (left_table) INNER JOIN (right_table) ON left_table.id = right_table.id;</pre>	<pre>SELECT * FROM Flug INNER JOIN Flughafen ON Flug.AbflugFH_ID = Flughafen.FH_ID;</pre>

Alias für Tabellen

Man kann Tabellen (sowie Spalten) ein Alias geben, um den Code übersichtlicher zu gestalten

Altes Beispiel	Neues Beispiel
<pre>SELECT * FROM Flug INNER JOIN Flughafen ON Flug.AbflugFH_ID = Flughafen.FH_ID;</pre>	<pre>SELECT c1.Preis, c2.Code, c1.FlugNr FROM Flug c1 INNER JOIN Flughafen c2 ON c1.AbflugFH_ID = c2.FH_ID;</pre>

Aufgaben – INNER JOIN

Aufgaben:

1. Lass dir jeden Flussnamen mit dessen Länge in einer Tabelle anzeigen. (Tabellen: Fluss, Gewaesser)
2. Lasse dir von jeder Hauptstadt (eines Landesteil) die Anzahl der Einwohner in dem dazugehörigen Landteil anzeigen. (Tabellen: Stadt, Landesteil)

Lösungen – INNER JOIN

Lösungen:

1. `SELECT c1.laenge, c2.name FROM Fluss c1 INNER JOIN
Gewaesser c2 ON c1.G_ID =c2.G_ID; --Amazonas 6518m`
2. `SELECT c1.Name, c2.Einwohner FROM Stadt c1 INNER JOIN
Landesteil c2 ON c1.S_ID =c2.Hauptstadt; --520 Zeilen`



Exkurs SQL Tutorial

SQL – Tutorial

- Einfache Lektionen
- Zahlreiche Übungen
- Roter Faden

<https://sql-tutorial.de/home/start.php>

Hinweis: Die Inhalte basieren auf dem SQL-Tutorial der LUO Darmstadt und sind kein offizielles Material von STADS e.V.

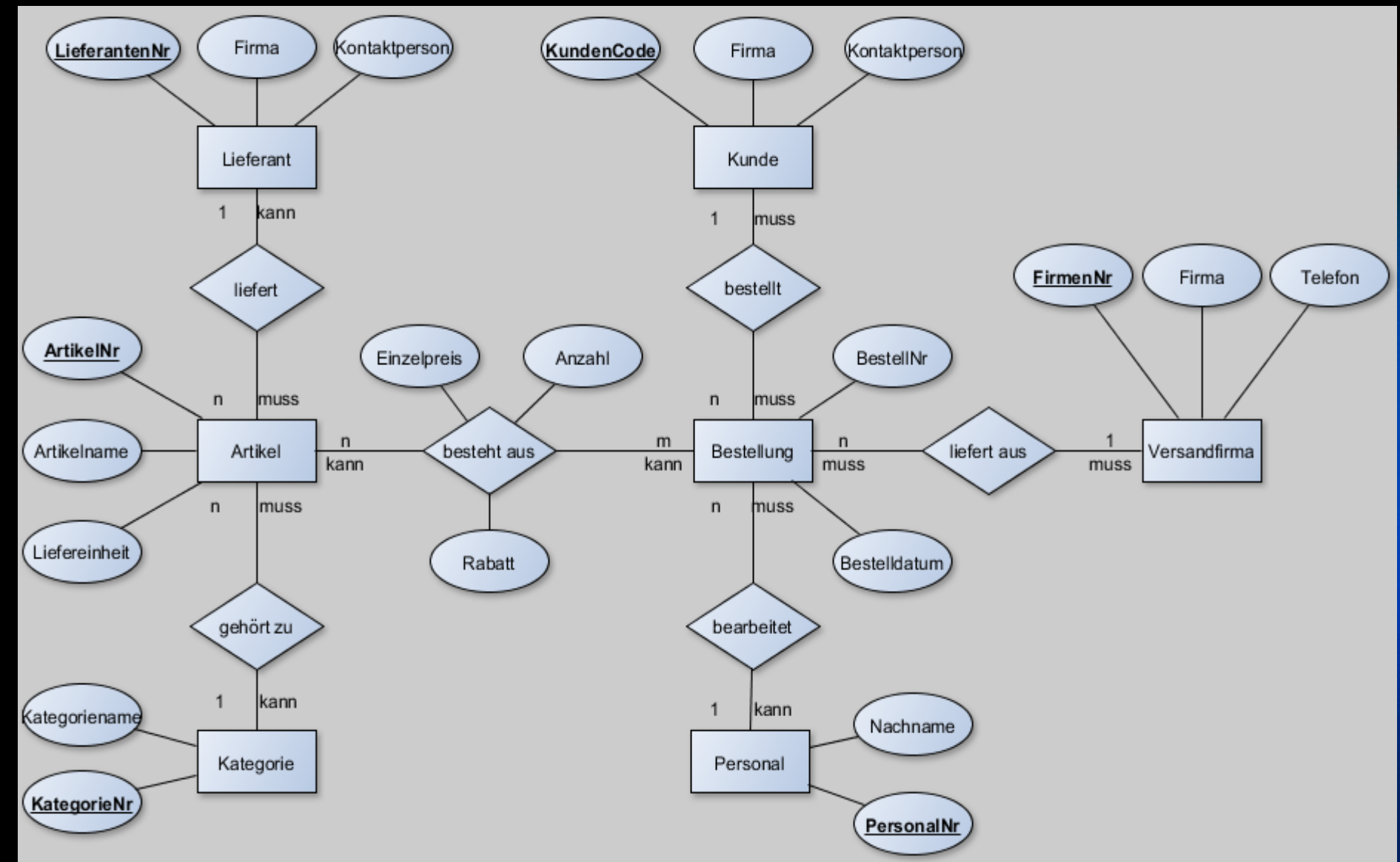
SQL – Tutorial

- Lektion 3: JOINS
(Die offiziellen Lösungen dieses Kapitels sind leider etwas umständlich)
- Aufgabe 9
- Welche Artikel hat der Kunde 'Richter Supermarkt' schon bestellt?

Lösung

```

SELECT Artikelname
FROM Artikel A
INNER JOIN Bestelldetails BD
    ON A.ArtikelNr = BD.ArtikelNr
INNER JOIN Bestellung B
    ON BD.BestellNr = B.BestellNr
INNER JOIN Kunde K
    ON B.KundenCode = K.KundenCode
WHERE Firma = 'Richter Supermarkt';
    
```





***Ausblick:
Daten einfügen
und ändern***

**YOU USE
SQL
UPDATE**

**YOU FORGET
SQL WHERE**

**37,052 ROWS
AFFECTED**



INSERT INTO

- Reihenfolge der Spalten wichtig!
- Pflichtspalten und Datentypen beachten

Syntax	Beispiel
<pre>INSERT INTO [tabelle] ([spalte1] , [spalte2]) VALUES ([wert1] , [wert2])</pre>	<pre>INSERT INTO Stadt (S_ID, Name, Einwohner) VALUES (9999, 'Teststadt', 50000);</pre>

UPDATE/SET

- **ACHTUNG:** Auf korrekte WHERE Bedingung achten

Syntax	Beispiel
UPDATE [tabelle] SET [spalte] = [wert] WHERE [bedingung]	UPDATE Stadt SET Einwohner = 60000 WHERE Name = 'Teststadt';

Hinweis: *UPDATE Stadt SET Einwohner = 60000; Würde ALLE Zeilen ändern*

DELETE

- **ACHTUNG:** Auf korrekte WHERE Bedingung achten

Syntax	Beispiel
DELETE FROM [tabelle] WHERE [bedingung]	DELETE FROM Stadt WHERE Name = 'Teststadt';

Hinweis: *DELETE ohne WHERE löscht alle Zeilen der Tabelle*



Ausblick: Nested SELECT

Senior dev: Haha that's weird it's
almost like the database is gone

The intern:



Verschachteltes SELECT

```
SELECT Name, Einwohner  
FROM Stadt  
WHERE Einwohner > (  
    SELECT AVG(Einwohner)  
    FROM Stadt  
);
```

	Name	Einwohner
1	Adelaide	987100
2	Aleppo	976727
3	Alma-Ata	1100000
4	Amman	777500
5	Ankara	2845689
6	Athen	885737
7	Bagdad	3200000
8	Baku	1740000
9	Baltimore	786775

Cheatsheet

- **ZUERST** Schema erkunden
 -
 - **DATEN ÄNDERN**
 - INSERT INTO t (sp1, sp2) VALUES (v1, v2);
 - UPDATE t SET sp = wert WHERE bedingung;
 - DELETE FROM t WHERE bedingung;
 -
 - **REIHENFOLGE DER KLAUSELN**
 - SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT
 -
 - **HÄUFIGE FEHLER**
 - • WHERE sp = NULL → IS NULL verwenden
 - • Texte in 'Anführungszeichen'
 - • UPDATE / DELETE immer mit WHERE!
 - • LIKE nur bei Textmuster, nicht bei =
- **ABFRAGEN**
 -
 - SELECT sp1, sp2 FROM tabelle; → Spalten wählen
 - SELECT * FROM tabelle LIMIT 10; → alle Spalten, begrenzt
 - SELECT DISTINCT sp FROM tabelle; → Duplikate entfernen
 - WHERE sp = 'Wert' / > Zahl → filtern
 - AND / OR → kombinieren
 - BETWEEN a AND b → Bereich
 - IN ('a','b','c') → Werteliste
 - IS NULL / IS NOT NULL → fehlende Werte
 - LIKE 'M%' → Textmuster
 - ORDER BY sp ASC/DESC → sortieren
 - COUNT(*) SUM AVG MIN MAX → Aggregation
 - GROUP BY sp HAVING bedingung → gruppieren
 - INNER JOIN t2 ON t1.sp = t2.sp → verbinden

 ***Thank you!***

Questions 